

Dossiê de Funcionamento

MYINC Social Media AI V2 - Arquitetura em 3 Camadas

Frontend Web na Vercel + Supabase central + Motor Local EXE

Status: revisado com testes de contrato V2 passando

Dossiê de Funcionamento - MYINC Social Media AI V2

Versão: v2.0.0-local-engine

Arquitetura: Web na Vercel + Supabase central + Motor Local EXE

Status da revisão: revisado por contrato estático, sintaxe dos módulos Node/Electron e testes V2.

1. Objetivo

A V2 foi desenhada como uma aplicação nova e individual em três camadas, usando o V11 somente como referência de UX, identidade visual e fluxos aproveitáveis. A regra principal é separar controle, armazenamento e processamento pesado.

- Frontend web: painel visual, login, operação, aprovação e controle.
- Supabase: banco, storage, fila, logs, prompts, regras, agendamento e Edge Functions leves.
- Motor local EXE: processamento pesado de texto, imagem, carrossel, qualidade e futuramente vídeo.

2. Regra de ouro aplicada

- O frontend não chama OpenAI.
- O frontend não contém service role.
- O frontend não contém chaves Meta.
- O frontend não processa imagens ou vídeos.
- Supabase não executa geração pesada.
- Edge Functions apenas enfileiram, montam prompt, salvam resultado do motor e publicam mídia pronta.
- O EXE local processa 1 job por vez.
- O publicador não finge sucesso: sem Meta configurado, mídia HTTPS pública ou aprovação, ele falha e registra log.

3. Estrutura entregue

myinc-v2-local-engine/	
apps/web/	Frontend Vercel
supabase/v2/	SQL seguro V2
supabase/functions/	Edge Functions leves
engine/	Motor local Node/Electron
electron/	Shell desktop/tray/autostart
docs/	Documentação técnica
tests/	Testes de contrato V2

4. Frontend Web - Vercel

O frontend fica em apps/web e usa apenas variáveis públicas:

```
VITE_SUPABASE_URL  
VITE_SUPABASE_ANON_KEY  
VITE_APP_URL  
VITE_APP_ENV
```

Telas previstas no painel:

- Dashboard
- Planejamento

- Aprovação
- Memória da Marca
- Cérebro da IA
- Biblioteca
- Estúdio Criativo
- Calendário
- Fila
- Motor Local
- Logs
- Configurações

Funções principais:

1. Login via Supabase Auth.
2. Criação de campanha teste com 5 posts.
3. Envio de posts para produção via Edge Function enqueue-generation.
4. Aprovação de posts.
5. Agendamento em publish_queue somente quando mídia pronta existe.
6. Visualização de fila, workers e logs.

5. Supabase - Banco, fila, storage e logs

Scripts em supabase/v2:

1. 00_BACKUP_RUNTIME_SECRETS.sql - cria backup seguro se a tabela existir.
2. 01_DROP_APP_TABLES_ONLY.sql - remove somente tabelas do app.
3. 02_CREATE_V2_SCHEMA.sql - cria schema V2 completo.
4. 03_CREATE_RLS_POLICIES.sql - habilita RLS e separa leitura do painel de escrita do worker.
5. 04_CREATE_STORAGE_BUCKETS.sql - cria buckets.
6. 05_SEED_MYINC_BRAIN.sql - cria usuário, marca MYINC, perfil e cérebro inicial.
7. 06_CREATE_RPC_QUEUE_FUNCTIONS.sql - cria RPCs da fila e locks.
8. 07_CREATE_PUBLISH_SCHEDULE.sql - modelo de agendamento leve.

Tabelas centrais:

- brands, brand_profiles
- ai_brain_rules, ai_prompt_templates
- campaigns, monthly_plans, post_ideas, posts, post_versions
- library_items, brand_assets, media_assets
- worker_devices, generation_jobs, generation_job_events
- publish_queue, publish_logs
- system_logs, app_settings

6. Edge Functions leves

Edge Functions entregues:

- `enqueue-generation`: recebe posts e cria jobs.
- `build-prompt`: monta prompt payload a partir do banco.
- `engine-register-worker`: registra o EXE local por chave de worker.
- `engine-save-result`: recebe resultado do EXE, salva asset, versão e completa job.
- `publish-scheduled-posts`: publica mídia já pronta na Meta.
- `admin-status`: retorna status sem vaziar secrets.

Pontos de segurança:

- `engine-save-result` exige `x-worker-device-key`.
- Worker é validado por hash.
- Resultado só é aceito se o job estiver processing e travado pelo worker correto.
- Publicação exige `PUBLISH_CRON_SECRET`.

7. Motor Local EXE

O motor local fica em `engine/` e é iniciado pelo Electron em `electron/`.

Fluxo:

1. Lê `.env.engine` no computador local.
2. Registra worker pelo Edge `engine-register-worker`.
3. Envia heartbeat.
4. Chama RPC `claim_next_generation_job`.
5. Processa 1 job por vez.
6. Envia resultado para `engine-save-result`.
7. Em caso de erro, chama `fail_generation_job`.
8. Atualiza status local em `.myinc-engine/status.json`.

Tipos de job:

- `content`: gera legenda, headline, CTA, hashtags e prompt de imagem.
- `image`: gera imagem, salva via Edge em `creative-media`.
- `carousel_page`: gera uma página por job e atualiza `carousel_media_urls`.
- `quality_check`: registra avaliação básica.
- `video`: falha com erro claro se provider real não estiver configurado.

8. Publicação automática

A publicação é separada da geração.

Fluxo:

1. Post precisa estar aprovado ou aguardando revisão.
2. Mídia precisa estar pronta e pública via HTTPS.
3. Registro entra em `publish_queue`.
4. Cron chama `publish-scheduled-posts`.
5. Edge Function publica na Meta.
6. Retorno bruto vai para `publish_logs.response_json`.
7. Erros reais ficam em `publish_queue.error_message` e `publish_logs`.

Formatos tratados:

- Imagem única: IMAGE
- Carrossel: CAROUSEL
- Reels/video: REELS

Observação: vídeo/Reels pode exigir polling de processamento da Meta. A função tenta publicar e, se a Meta indicar que ainda não está pronto, registra erro real; não simula sucesso.

9. Segurança

Validações aplicadas:

- Nenhuma `SUPABASE_SERVICE_ROLE_KEY` no frontend.
- Nenhuma chave OpenAI no frontend.
- Nenhuma chave Meta no frontend.
- Motor local não usa service role.
- Worker usa anon key + chave de worker + RPCs/Edge validadas.
- Backup de runtime secrets antes de reset.
- Reset não usa reset total do schema.
- Buckets e tabelas são criados por scripts específicos.

10. Comandos de validação

Comando principal:

```
npm run v2:verify
```

Valida:

- Sintaxe dos arquivos principais do motor.
- Estrutura de 3 camadas.
- Ausência de secrets no frontend.
- Contrato SQL da fila.
- Contrato do motor local.
- Contrato das Edge Functions.

Também foi validado:

```
node --check electron/main.cjs
node --check electron/tray.cjs
node --check electron/engine-process.cjs
```

Resultado final da revisão:

```
v2-architecture ok
v2-no-secret-leak ok
v2-sql-contract ok
v2-engine-contract ok
v2-edge-contract ok
```

11. Como aplicar

Ordem recomendada:

1. Criar branch v2/local-engine-architecture.
2. Copiar o pacote para o repositório.
3. Rodar `npm run install:all`.
4. Configurar variáveis públicas da Vercel.
5. Aplicar SQLs em ordem de 00 a 07.
6. Fazer deploy das Edge Functions.
7. Configurar Edge Secrets.
8. Criar `.env.engine` no PC local.
9. Rodar `npm run engine:dev`.
10. Rodar `npm run web:dev`.
11. Criar campanha teste com 5 posts.
12. Enviar para produção.
13. Confirmar worker online e jobs processados.
14. Agendar post com mídia pronta.
15. Testar publicador.

12. Variáveis necessárias

Frontend/Vercel:

```
VITE_SUPABASE_URL
VITE_SUPABASE_ANON_KEY
VITE_APP_URL
VITE_APP_ENV
```

Edge/Supabase Secrets:

```
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
WORKER_DEVICE_KEY
PUBLISH_CRON_SECRET
META_PAGE_ACCESS_TOKEN
META_PAGE_ID
META_INSTAGRAM_BUSINESS_ID
META_GRAPH_VERSION opcional
```

Motor local `.env.engine`:

```
SUPABASE_URL
SUPABASE_ANON_KEY
SUPABASE_WORKER_DEVICE_KEY
OPENAI_API_KEY
ENGINE_WORKER_NAME
ENGINE_POLL_INTERVAL_MS
ENGINE_MAX_CONCURRENCY=1
ENGINE_AUTO_START=true
OPENAI_TEXT_MODEL=gpt-5.5
OPENAI_IMAGE_MODEL=gpt-image-2
OPENAI_IMAGE_QUALITY=high
OPENAI_IMAGE_FORMAT=png
```

13. Limitações honestas

Não é possível afirmar funcionamento 100% em produção sem executar no seu Supabase real, com:

- Auth configurado.
- Edge Secrets reais.
- Buckets criados.
- OpenAI API válida no PC local.
- Conta Meta aprovada e permissões corretas.
- Mídias públicas por HTTPS.

O que foi garantido nesta revisão:

- Arquitetura separada corretamente.
- Contratos internos coerentes.
- Testes V2 passando.
- Sem geração pesada no frontend/Edge.
- Sem vazamento de secrets no frontend.
- Motor local com fluxo de claim, processamento, fail e save-result.
- Publicador sem sucesso simulado.

14. Checklist de produção

- ☐ Rodar SQL 00 antes de qualquer reset.
- ☐ Confirmar que `runtime_secrets` foi preservada se existir.
- ☐ Aplicar SQL 01 a 07 na ordem.
- ☐ Criar usuário Rodrigo no Supabase Auth com o mesmo e-mail da seed.
- ☐ Configurar Vercel com apenas variáveis públicas.
- ☐ Deploy Edge Functions.
- ☐ Configurar Edge Secrets.
- ☐ Criar `.env.engine` no PC.
- ☐ Rodar `npm run engine:dev` e confirmar heartbeat.
- ☐ Criar 5 posts de teste.
- ☐ Enviar para produção.
- ☐ Confirmar jobs em `generation_jobs`.

- ☐ Confirmar mídia em creative-media.
- ☐ Confirmar media_assets e post_versions.
- ☐ Aprovar/agendar.
- ☐ Testar publicação real com uma imagem simples antes de carrossel/reels.